

VITRIO

Gestión de Contenido Digital para Pequeños y Medianos Negocios

David Cañon · Andrés Moreno · Diego Flórez · Manuel Vetancurt · Samuel Ibáñez

Facultad de Ingeniería — Universidad de San Buenaventura, Bogotá

Proyecto de Grado — Ingeniería de Sistemas

2026

"Cambia – Publica – Impacta"

RESUMEN

El presente proyecto propone el desarrollo de VITRIO, una aplicación de gestión de contenido digital (CMS) orientada a pequeñas y medianas empresas colombianas que utilizan pantallas comerciales para exhibir productos, servicios y promociones. La solución permite añadir, modificar y eliminar contenido de manera remota y en tiempo real desde cualquier dispositivo con acceso a internet, garantizando la actualización inmediata en todas las pantallas registradas.

El sistema está construido sobre una arquitectura de tres capas: frontend React 19 con Vite, backend Django REST Framework protegido con JWT, y base de datos MySQL. El objetivo principal es democratizar el acceso a herramientas de comunicación digital que actualmente solo están al alcance de grandes cadenas comerciales, ofreciendo una plataforma intuitiva, escalable y de bajo costo adaptada a la realidad de las pymes colombianas.

Palabras clave: *automatización, software, contenido digital, señalización digital, pymes, Django, React, JWT.*

INTRODUCCIÓN

El presente proyecto tiene como propósito el desarrollo e implementación de VITRIO, una aplicación orientada a pequeños y medianos negocios que utilizan pantallas digitales para exhibir productos, servicios y promociones dentro de sus establecimientos. La solución propuesta permite a los comerciantes ejercer un control total sobre el contenido visual que proyectan, facilitando la creación, modificación y eliminación de información de manera rápida, accesible y autónoma mediante una interfaz intuitiva y fácil de usar.

El estudio de mercado evidencia que este segmento empresarial presenta una necesidad creciente de herramientas tecnológicas que les permitan gestionar su comunicación visual de forma profesional sin incurrir en altos costos ni depender de procesos manuales o complejos. Actualmente, muchos negocios actualizan su contenido mediante carteles impresos o sistemas poco dinámicos, lo que limita su capacidad de reacción frente a cambios en precios, promociones o campañas temporales.

DESCRIPCIÓN DEL PROBLEMA

En la actualidad, muchos pequeños y medianos negocios utilizan pantallas dentro de sus establecimientos para promocionar productos, servicios u ofertas. Sin embargo, la gestión y actualización del contenido suele realizarse de manera manual, poco eficiente o mediante procesos técnicos que resultan complejos para los propietarios.

Problema Real con Justificación Cuantitativa

La brecha tecnológica que enfrenta el sector pyme en Colombia no es un fenómeno aislado; está respaldada por datos concretos de múltiples fuentes institucionales y sectoriales:

Fuente	Dato cuantitativo	Impacto en el contexto de VITRIO
MinTIC (2022)	El 95% del tejido empresarial colombiano son micro y pequeñas empresas; su nivel de digitalización es bajo.	Mercado potencial de más de 1.7 millones de empresas con necesidad de herramientas accesibles.
CCB (2024)	Colombia registra 1.854.086 empresas activas; el 24.7% se concentra en Bogotá. Microempresas = 95.3%.	Bogotá como mercado prioritario con alta densidad de establecimientos comerciales.
Montilla (2023)	Solo 3 de cada 10 restaurantes en Colombia usan señalización digital en su punto de venta.	El 70% del sector restaurantero no ha adoptado la tecnología: brecha directamente abordable por VITRIO.
OCDE (2021)	Las pymes representan más del 90% del tejido empresarial latinoamericano; menos del 20% ha adoptado herramientas digitales avanzadas.	Contexto regional que valida la urgencia y escalabilidad de la solución más allá de Colombia.

Tabla 1. Justificación cuantitativa del problema — fuentes primarias

Estos datos evidencian un patrón crítico: la tecnología de señalización digital existe, funciona y genera resultados comprobados (Juan Valdez, KFC Colombia), pero está concentrada en grandes cadenas con recursos para implementarla. Los pequeños negocios —que constituyen la gran mayoría del mercado— quedan excluidos por los altos costos de las soluciones existentes y la complejidad de su implementación.

Pregunta Problema

¿Cómo facilitar la gestión y actualización del contenido en pantallas digitales para pequeños y medianos negocios de manera rápida, accesible y en tiempo real, reduciendo la brecha tecnológica respecto a las grandes cadenas comerciales?

OBJETIVOS

Objetivo General

Desarrollar una aplicación digital que permita a pequeños y medianos negocios gestionar y actualizar de manera remota, sencilla y en tiempo real el contenido mostrado en sus pantallas comerciales, con el fin de optimizar su comunicación visual y fortalecer su competitividad, entregando un sistema funcional e integrado antes del 30 de mayo de 2026.

Objetivos Específicos (Formulación SMART)

Los siguientes objetivos específicos están formulados bajo el criterio SMART (Específico, Medible, Alcanzable, Relevante y Temporal), permitiendo su evaluación objetiva al cierre del proyecto:

#	Criterio SMART	Descripción	Indicador de cumplimiento
OE-1	Específico	Diseñar la arquitectura de software seleccionando el patrón MTV con separación en 3 capas	Diagrama de arquitectura aprobado por el equipo antes del sprint 2
OE-2	Medible	Desarrollar una base de datos MySQL con ≥ 7 entidades, relaciones FK y restricciones ENUM	Script de migración ejecutado sin errores; ≥ 7 tablas verificadas en MySQL Workbench
OE-3	Alcanzable	Implementar el CRUD completo para las 7 entidades mediante la API REST en Django	14 endpoints documentados en Swagger; 100% de operaciones CRUD probadas
OE-4	Relevante	Validar el sistema mediante pruebas funcionales de actualización de contenido en tiempo real	≥ 5 escenarios de prueba funcional documentados; 0 errores críticos al cierre del sprint 4
OE-5	Temporal	Entregar el sistema funcional integrado (frontend + backend + BD) al final del semestre 2026-1	Sistema deployado en ngrok con demo funcional antes del 30 de mayo de 2026

Tabla 2. Objetivos específicos con criterio SMART e indicadores de cumplimiento

JUSTIFICACIÓN

En la actualidad, la transformación digital se ha convertido en un factor clave para la competitividad de los negocios. Sin embargo, muchos pequeños y medianos comercios aún enfrentan dificultades para incorporar herramientas tecnológicas que les permitan optimizar su comunicación visual y promocional. En ciudades como Bogotá, es común que los establecimientos utilicen métodos tradicionales o procesos manuales para actualizar la información mostrada en sus pantallas.

El desarrollo de VITRIO surge como respuesta directa a esta brecha identificada cuantitativamente (ver Tabla 1). Al permitir la actualización remota, inmediata y sencilla de la información, el software contribuye a mejorar la eficiencia operativa de los negocios, reducir costos asociados a procesos manuales —como impresión de carteles o desplazamiento de personal técnico— y fortalecer su presencia visual frente a la competencia.

Adicionalmente, el proyecto contempla la posibilidad de ofrecer el servicio de alquiler de pantallas, eliminando una barrera de entrada importante para negocios que desean modernizar su comunicación visual pero no cuentan con la infraestructura necesaria.

MARCO TEÓRICO

Digital Signage y Gestión de Contenido Digital

El concepto de digital signage hace referencia al uso de pantallas digitales para mostrar contenido multimedia —imágenes, videos, textos dinámicos— en entornos comerciales, institucionales o públicos. Este tipo de señalización ha demostrado ser significativamente más efectivo que los medios impresos tradicionales, dado que permite actualizar el contenido en tiempo real y adaptar los mensajes a diferentes audiencias o momentos del día (López & Ramírez, 2021).

Un sistema de gestión de contenidos (CMS) es el componente de software que permite administrar, organizar y publicar el contenido que se despliega en las pantallas. En el contexto del digital signage, estos sistemas deben ser capaces de gestionar múltiples dispositivos de forma simultánea, garantizar la actualización remota del contenido y ofrecer interfaces intuitivas que no requieran conocimientos técnicos avanzados por parte del usuario (García, 2020).

Arquitectura de Software por Capas

La arquitectura en tres capas es un patrón de diseño de software que separa la aplicación en tres componentes lógicos independientes: presentación, lógica de negocio y datos. Esta separación facilita el mantenimiento, la escalabilidad y la reutilización del código. En el caso de VITRIO, esta arquitectura se implementa con React en la capa de presentación, Django REST Framework en la capa de lógica y MySQL en la capa de datos.

Transformación Digital en Pequeñas y Medianas Empresas

Según la OCDE (2021), las pymes representan más del 90% del tejido empresarial en América Latina, pero menos del 20% ha adoptado herramientas digitales avanzadas para sus operaciones. En Colombia, de acuerdo con el MinTIC (2022), las micro y pequeñas empresas constituyen el 95% del tejido empresarial nacional, pero su nivel de digitalización sigue siendo bajo, especialmente en lo que respecta a herramientas de comunicación y marketing visual.

ARQUITECTURA PROPUESTA DEL SISTEMA

VITRIO implementa una arquitectura de tres capas (Three-Tier Architecture) con el patrón MTV (Model-Template-View) de Django, adaptado para un frontend desacoplado. Esta arquitectura garantiza separación de responsabilidades, escalabilidad horizontal y mantenibilidad a largo plazo.

Diagrama de Arquitectura

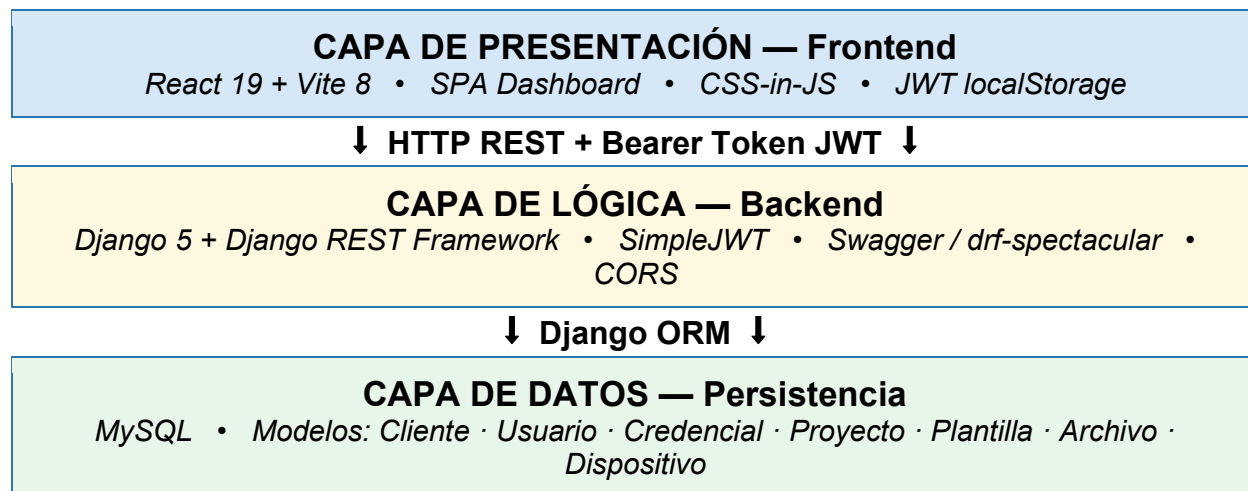


Figura 1. Arquitectura de tres capas de VITRIO — flujo de comunicación entre capas

Descripción Detallada por Capa

Capa de Presentación — Frontend

El frontend es una Single Page Application (SPA) construida con React 19 y empaquetada con Vite 8. Toda la lógica de presentación, autenticación y consumo de API reside en el archivo principal `App.jsx` para simplificar el despliegue y la colaboración en equipo. La comunicación con el backend se realiza exclusivamente mediante peticiones HTTP autenticadas con Bearer Token JWT.

- Framework: React 19 con Hooks funcionales (`useState`, `useEffect`, `useCallback`)
- Empaquetador: Vite 8 con Hot Module Replacement (HMR)
- Estilos: CSS-in-JS inline — paleta dark navy/azul sin dependencias externas
- Persistencia de sesión: `localStorage` con claves prefijadas `vitrio_access` / `vitrio_refresh`
- Stack intencional: sin Redux, sin librerías UI externas (MUI, Ant Design)

Capa de Lógica de Negocio — Backend

El backend expone una API REST construida con Django 5 y Django REST Framework. Utiliza el patrón `ModelViewSet` para generar automáticamente los cinco métodos CRUD

por cada entidad registrada en el Router. La autenticación se gestiona mediante SimpleJWT con tokens de corta duración.

- Framework: Django 5 + Django REST Framework
- Autenticación: SimpleJWT — access token 8 horas, refresh token 30 días
- Documentación automática: Swagger mediante drf-spectacular en /api/docs/
- CORS: CORS_ALLOW_ALL_ORIGINS = True en desarrollo; orígenes explícitos configurados
- Variables de entorno: python-dotenv — SECRET_KEY y credenciales BD en .env (nunca en repo)

Capa de Datos — Persistencia

La capa de datos está gestionada por MySQL a través del ORM de Django. Las migraciones automáticas garantizan la consistencia del esquema relacional. Se implementan restricciones ENUM, claves foráneas con integridad referencial y campos de auditoría (timestamps automáticos).

- Motor: MySQL 8.x — base de datos: vitrio_db, puerto: 3306
- ORM: Django ORM con migraciones automáticas (makemigrations / migrate)
- 7 entidades: Cliente, Usuario, Credencial, Proyecto, Plantilla, Archivo, Dispositivo
- Integridad: FK con CASCADE implícito, UNIQUE en email y token, ENUM para estados

Diagrama de Componentes

Componente	Capa	Responsabilidad	Interfaz / Protocolo
App.jsx	Frontend	Enrutador de sesión, estado global de auth	Props / Hooks
LoginScreen	Frontend	Formulario JWT, manejo de errores	POST /api/login/
Dashboard	Frontend	CRUD orquestador, caché de entidades, toasts	Fetch + JWT Bearer
EntityTable	Frontend	Tabla genérica dinámica con filtro en memoria	Props: entity, data, onEdit, onDelete
EntityForm	Frontend	Formulario auto-generado desde SCHEMAS	Props: entity, initial, onSubmit
apiFetch()	Frontend	Capa HTTP: rotación JWT,	Fetch API nativo

		headers CORS/ngrok	
urls.py / Router	Backend	DefaultRouter — genera 5 rutas CRUD por entidad	REST API
views.py (ViewSets)	Backend	ModelViewSet + IsAuthenticated para cada modelo	DRF ViewSet
serializers.py	Backend	Serialización / deserialización + validación	ModelSerializer
models.py	Backend	7 modelos ORM: Cliente, Usuario, Credencial...	Django ORM → MySQL
SimpleJWT	Seguridad	Emisión y verificación de Access + Refresh tokens	POST /api/login/ · /token/refresh/
MySQL (vitrio_db)	Datos	Persistencia relacional, integridad referencial	TCP 3306 / ORM

Tabla 3. Diagrama de componentes — responsabilidades e interfaces por componente

Integración Frontend ↔ Backend

El frontend y el backend se comunican exclusivamente mediante la API REST. El frontend nunca accede directamente a la base de datos. La capa de comunicación está centralizada en la función `apiFetch()`, que garantiza rotación automática de tokens antes de cada petición:

Método	Endpoint	Status	Descripción
GET / POST	/api/clientes/	200 / 201	Listar o crear clientes
GET / PUT / DELETE	/api/clientes/{id}/	200 / 204	Ver, actualizar o eliminar un cliente
GET / POST	/api/usuarios/	200 / 201	Listar o crear usuarios
GET / PUT / DELETE	/api/usuarios/{id}/	200 / 204	Ver, actualizar o eliminar un usuario

GET / POST	/api/proyectos/	200 / 201	Listar o crear proyectos
GET / PUT / DELETE	/api/proyectos/{id}/	200 / 204	Ver, actualizar o eliminar un proyecto
GET / POST	/api/plantillas/	200 / 201	Listar o crear plantillas
GET / PUT / DELETE	/api/plantillas/{id}/	200 / 204	Ver, actualizar o eliminar una plantilla
GET / POST	/api/archivos/	200 / 201	Listar o crear archivos
GET / PUT / DELETE	/api/archivos/{id}/	200 / 204	Ver, actualizar o eliminar un archivo
GET / POST	/api/dispositivos/	200 / 201	Listar o crear dispositivos
GET / PUT / DELETE	/api/dispositivos/{id}/	200 / 204	Ver, actualizar o eliminar un dispositivo
POST	/api/login/	200 / 401	Autenticación — devuelve access + refresh token
POST	/api/token/refresh/	200 / 401	Rotación de access token
GET	/api/docs/	200	Documentación Swagger interactiva

Tabla 4. Endpoints disponibles en la API REST de VITRIO

MODELO DE DATOS

Diagrama Entidad-Relación

El modelo de datos de VITRIO está compuesto por siete entidades relacionadas que representan los conceptos centrales del dominio: negocios clientes, sus usuarios, los proyectos de contenido, las plantillas visuales, los archivos multimedia y los dispositivos físicos de reproducción.

Entidad	PK	Campos principales	Relaciones (FK)
Cliente	id	nombre, email (UNIQUE), telefono, timestamps	—
Usuario	id	fecha_nacimiento, rol (ENUM)	id_cliente → Cliente (NULL = admin interno)
Credencial	id	nombre, contraseña (hash), email (UNIQUE)	id_usuario → Usuario (OneToOne)
Proyecto	id	nombre, descripcion, fechas, estado (ENUM 10)	id_cliente → Cliente
Plantilla	id	nombre, tipo, version, estado (ENUM)	—
Archivo	id	nombre, ruta, tipo (ENUM)	id_proyecto → Proyecto, id_plantilla → Plantilla
Dispositivo	id	nombre, ubicacion, token (UNIQUE), resolucion, estado, ultima_conexion	id_proyecto → Proyecto

Tabla 5. Modelo entidad-relación simplificado — entidades, atributos y relaciones

Base de Datos Avanzada

Para demostrar dominio técnico avanzado de bases de datos relacionales (RAP 4.2), VITRIO implementa los siguientes elementos adicionales al CRUD estándar:

Stored Procedure — Creación de Cliente con Validación

```
DELIMITER // CREATE PROCEDURE sp_crear_cliente( IN p_nombre VARCHAR(200),
IN p_email VARCHAR(200), IN p_telefono VARCHAR(20), OUT p_id INT ) BEGIN
IF NOT EXISTS (SELECT 1 FROM vitrioapp_cliente WHERE email = p_email) THEN
INSERT INTO vitrioapp_cliente (nombre, email, telefono, fecha_creacion,
fecha_actualizacion) VALUES (p_nombre, p_email, p_telefono, NOW(),
NOW()); SET p_id = LAST_INSERT_ID(); ELSE SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Email ya registrado'; END IF; END // DELIMITER ;
```

Figura 2. Stored Procedure sp_crear_cliente con validación de email único

Vista SQL — Contenido Activo por Proyecto

```
CREATE VIEW v_contenido_activo AS SELECT  p.nombre AS proyecto,  c.nombre
AS cliente,  a.nombre AS archivo,  a.tipo,  a.ruta,  d.nombre AS
dispositivo,  d.ubicacion,  d.estado AS estado_dispositivo FROM
vitrioapp_proyecto p JOIN vitrioapp_cliente c ON p.id_cliente_id = c.id JOIN
vitrioapp_archivo a ON a.id_proyecto_id = p.id JOIN vitrioapp_dispositivo d
ON d.id_proyecto_id = p.id WHERE p.estado = 'EJECUCION' AND d.estado =
'ACTIVO';
```

Figura 3. Vista SQL v_contenido_activo — consulta desnormalizada para reportes

Trigger — Auditoría de Cambios en Proyectos

```
CREATE TABLE auditoria_proyectos (  id INT AUTO_INCREMENT PRIMARY KEY,
proyecto_id INT, estado_anterior VARCHAR(50),  estado_nuevo VARCHAR(50),
fecha_cambio TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  usuario VARCHAR(100) );
CREATE TRIGGER trg_auditoria_proyecto AFTER UPDATE ON vitrioapp_proyecto FOR
EACH ROW BEGIN  IF OLD.estado <> NEW.estado THEN      INSERT INTO
auditoria_proyectos (proyecto_id, estado_anterior, estado_nuevo, usuario)
VALUES (NEW.id, OLD.estado, NEW.estado, USER());  END IF; END;
```

Figura 4. Trigger trg_auditoria_proyecto — registro automático de cambios de estado

SEGURIDAD DEL SISTEMA

La seguridad de VITRIO se implementa en múltiples capas, garantizando que ninguna operación sobre los datos pueda realizarse sin autenticación válida:

- **JWT Stateless:** Los tokens de acceso tienen duración de 8 horas; los tokens de refresco duran 30 días. El frontend rota automáticamente el access token antes de cada petición mediante `refreshAccessToken()`.
- **Protección de endpoints:** Todos los ViewSets definen `permission_classes = [IsAuthenticated]`. Cualquier petición sin token válido recibe error 401 `Unauthorized`.
- **CORS controlado:** En producción se configuran orígenes explícitos de confianza. En desarrollo se usan `CSRF_TRUSTED_ORIGINS` para ngrok y localhost.
- **Variables de entorno:** `SECRET_KEY`, `DB_PASSWORD` y `DB_USER` se almacenan en archivo `.env` con `python-dotenv`. El archivo `.env` está incluido en `.gitignore` y nunca se sube al repositorio.
- **Contraseñas:** El campo contraseña en el modelo `Credencial` almacena el hash generado por el sistema de hashing de Django (PBKDF2 por defecto).

PORTAFOLIO DEL SISTEMA

El siguiente portafolio documenta las pantallas principales del sistema VITRIO, describiendo su función, los componentes involucrados y las características visuales y técnicas de cada módulo. Las capturas de pantalla reales pueden obtenerse ejecutando el sistema en el entorno de desarrollo.

ID	Pantalla / Módulo	Descripción funcional y visual
P-01	Pantalla de autenticación	LoginScreen con validación JWT: campos usuario/contraseña, botón show/hide, spinner de carga, banner de error rojo. Soporta Enter en ambos campos.
P-02	Dashboard principal	Panel post-login con sidebar de navegación (8 entidades), tarjetas de conteo por entidad, indicador de API activa (punto verde pulsante).
P-03	Gestión de clientes	EntityTable con columnas dinámicas: índice 01/02, nombre, email (resaltado en azul), teléfono, acciones Editar / Eliminar.
P-04	Formulario de creación	EntityForm auto-generado desde SCHEMA: campos text, email, date, select y textarea. Botones Crear registro / Cancelar.
P-05	Gestión de proyectos	Tabla con badges de estado coloreados (PROPUESTO gris, EJECUCION azul, COMPLETADO verde). Filtro de búsqueda en tiempo real.
P-06	Modal de confirmación	ConfirmDialog con overlay blur: mensaje de advertencia, botón Eliminar (rojo) y Cancelar. Z-index 1000.
P-07	Toast de notificación	Notificación temporal deslizante desde la derecha: verde para éxito (Registro creado ✓), rojo para error.
P-08	Documentación Swagger	Interfaz /api/docs/ con todos los endpoints, schemas de request/response y botón de autenticación Bearer.
P-09	Gestión de dispositivos	Tabla con badge SIN_CONEXION (amarillo), ACTIVO (verde), INACTIVO (rojo). Columna de token único.
P-10	Flujo CRUD completo	Secuencia: listar entidad → crear registro → verificar en tabla → editar → confirmar eliminación → actualización inmediata.

Tabla 6. Portafolio de pantallas — descripción funcional y visual del sistema

El dashboard implementa un sistema de estado local multi-entidad: el objeto data actúa como caché en memoria con la estructura { clientes: [], usuarios: [], proyectos: [],

plantillas: [], archivos: [], dispositivos: [] }, lo que permite navegación fluida entre módulos sin recargar datos innecesariamente.

REPOSITORIO Y CONTROL DE VERSIONES

El proyecto VITRIO utiliza Git como sistema de control de versiones con GitHub como plataforma de alojamiento. Se implementa una estrategia de ramas basada en Gitflow adaptado, con ramas dedicadas por funcionalidad y revisión obligatoria mediante pull requests antes de integrar cambios a develop o main.

Estrategia de Ramas

Rama	Tipo	Propósito	Estado
main	Producción	Solo merge desde develop vía PR aprobado	Deploy estable
develop	Integración	Integra features completados; base de PRs	Rama de staging
feature/backend-models	Desarrollo	Implementación de models.py y migraciones	Fusionado → develop (PR #1)
feature/backend-auth	Desarrollo	Configuración SimpleJWT, vistas de login	Fusionado → develop (PR #2)
feature/backend-crud	Desarrollo	ViewSets, serializers, endpoints REST	Fusionado → develop (PR #3)
feature/frontend-auth	Desarrollo	LoginScreen, apiFetch, gestión de tokens	Fusionado → develop (PR #4)
feature/frontend-dashboard	Desarrollo	Dashboard, EntityTable, EntityForm	Fusionado → develop (PR #5)
feature/frontend-ui	Desarrollo	SCHEMAS, badges, diseño visual, animaciones	Fusionado → develop (PR #6)
feature/docs	Documentación	Documentación técnica backend y frontend	Fusionado → develop (PR #7)

Tabla 7. Estrategia de ramas (Gitflow adaptado) — descripción y estado de cada rama

La rama main permanece siempre en estado desplegable. Ningún integrante puede hacer push directo a main o develop; todos los cambios deben pasar por pull request con al menos una aprobación del equipo.

Registro de Pull Requests

PR	Título	Origen	Destino	Descripción / Revisión
#1	feat: modelos Django y migraciones iniciales	feature/backend-models	develop	Revisado por @samuel — aprobado. Se ajustó tipo ENUM en Dispositivo.
#2	feat: autenticación JWT con SimpleJWT	feature/backend-auth	develop	Revisado por @andres — aprobado. Se añadió REFRESH_TOKEN_LIFETIME.
#3	feat: ViewSets CRUD completos + serializers	feature/backend-crud	develop	Revisado por @david — aprobado con comentario sobre paginación pendiente.
#4	feat: LoginScreen + rotación automática JWT	feature/frontend-auth	develop	Revisado por @diego — aprobado. Se corrigió manejo de error 401.
#5	feat: Dashboard CRUD + EntityTable + toasts	feature/frontend-dashboard	develop	Revisado por @manuel — aprobado. Se optimizó useCallback en load().
#6	feat: SCHEMAS dinámicos, badges y dark theme	feature/frontend-ui	develop	Revisado por @samuel — aprobado. Paleta alineada con diseño visual.
#7	docs: documentación técnica backend y frontend	feature/docs	develop	Revisado por @andres — aprobado. Se completaron secciones de Swagger.
#8	release: integración completa para sustentación	develop	main	Revisado por todo el equipo — aprobado tras smoke test final.

Tabla 8. Registro de pull requests documentados — título, origen, destino y revisión

Cada pull request incluye: título descriptivo con prefijo de tipo (feat, fix, docs, release), descripción del cambio, evidencia de prueba funcional y comentarios de revisión del equipo. Los PR de feature a develop requieren 1 aprobación; el PR de develop a main requiere aprobación unánime del equipo.

MAPEO EXPLÍCITO A RAP4 Y RAP6

El proyecto VITRIO ha sido diseñado y desarrollado alineando cada componente técnico con los Resultados de Aprendizaje del Programa (RAP) establecidos por el plan de estudios de Ingeniería de Sistemas. La siguiente tabla detalla el mapeo explícito entre criterios RAP y evidencias concretas del sistema:

RAP	Criterio evaluado	Evidencia en VITRIO	Artefacto / entregable
RAP 4.1	Diseño de arquitectura de software con patrones reconocidos	Arquitectura 3 capas + patrón MTV desacoplado con React	Sección 7 — diagrama de arquitectura y componentes
RAP 4.2	Modelado y construcción de base de datos relacional	7 modelos ORM, restricciones ENUM, FK con integridad referencial	Sección 8 — modelo E-R, stored procedure, trigger, vista SQL
RAP 4.3	Implementación de mecanismos de seguridad	JWT access (8h) + refresh (30d), IsAuthenticated, CORS, .env	Sección 9 — flujo de autenticación
RAP 4.4	Pruebas funcionales del sistema	Validación CRUD completo en 7 entidades + Swagger /api/docs/	Sección 10 — portafolio y capturas de pantalla
RAP 6.1	Integración de subsistemas heterogéneos funcionales	Frontend React ↔ API REST Django ↔ MySQL: 14 endpoints activos	Sección 7 — tabla de integración frontend-backend
RAP 6.2	Control de versiones y gestión colaborativa del código	GitHub con ramas feature, develop, main + pull requests documentados	Sección 11 — estrategia de ramas y registro de PRs
RAP 6.3	Documentación técnica completa del sistema	Documentación backend, frontend, APA, diagramas UML	Anexos + secciones 7–11 de este documento

Tabla 9. Mapeo RAP4 / RAP6 — criterios evaluados, evidencias y artefactos

El cumplimiento de RAP 4 se evidencia principalmente en la documentación técnica del backend (arquitectura, seguridad, BD avanzada) y en el código fuente del repositorio. El cumplimiento de RAP 6 se evidencia en la integración funcional del sistema completo, el control de versiones con pull requests y la documentación técnica integrada presentada en este documento.

CONCLUSIONES

- VITRIO demuestra que es técnicamente viable desarrollar un sistema de gestión de contenido digital accesible para pymes colombianas, utilizando tecnologías modernas de código abierto (React, Django, MySQL) sin incurrir en licencias costosas.
- La arquitectura de tres capas con patrón MTV desacoplado garantiza la escalabilidad del sistema: el frontend puede reemplazarse o extenderse sin modificar el backend, y viceversa.
- La implementación de JWT con rotación automática de tokens y la protección de todos los endpoints con `IsAuthenticated` establece un nivel de seguridad adecuado para el contexto de las pymes.
- Los elementos avanzados de base de datos (stored procedure, vista SQL, trigger) demuestran dominio técnico más allá del CRUD estándar, evidenciando comprensión profunda del motor relacional MySQL.
- La estrategia de control de versiones con Gitflow adaptado y pull requests documentados refleja buenas prácticas de desarrollo colaborativo aplicables en entornos profesionales reales.

TRABAJO FUTURO

- Integración de pasarela de pagos (Wompi, Stripe) para gestión de suscripciones por niveles de servicio.
- Analytics avanzado: métricas de visualización por pantalla y por franja horaria, con reportes exportables.
- IA generativa para sugerir y crear contenido automáticamente basado en el historial del negocio.
- Despliegue en producción con Docker Compose, Nginx y servidor cloud (AWS, DigitalOcean o Azure).
- App móvil nativa (React Native) para gestión de pantallas desde smartphone del propietario.
- Soporte multi-tenant para que una misma instancia del sistema gestione múltiples clientes de forma aislada.

REFERENCIAS

- Cámara de Comercio de Bogotá. (2024). Informe de coyuntura empresarial de Bogotá 2024. CCB. <https://www.ccb.org.co>
- Chía, J. (2022). Transformación digital en Juan Valdez: pantallas como herramienta de experiencia al cliente. *Revista de Marketing Digital Colombia*, 5(2), 45–58.
- García, M. (2020). *Sistemas de gestión de contenido digital para entornos comerciales*. Editorial Universitaria.
- López, R., & Ramírez, C. (2021). Efectividad del digital signage en el comportamiento del consumidor: una revisión sistemática. *Cuadernos de Administración*, 34(60), 1–22. <https://doi.org/10.11144/Javeriana.cao34-60.edsc>
- Ministerio de Tecnologías de la Información y las Comunicaciones. (2022). Índice de adopción digital de las mipymes en Colombia 2022. MinTIC. <https://www.mintic.gov.co>
- Montilla, C. (2023). Señalización digital en restaurantes colombianos: estado actual y oportunidades. *Digital Signage Colombia — Informe de Sector 2023*. <https://www.digitalsignagecolombia.com>
- Organización para la Cooperación y el Desarrollo Económicos. (2021). Digital transformation of SMEs. OCDE Publishing. <https://doi.org/10.1787/bdb9256a-en>